

Teaching Guidelines for
MERN Stack for FinTech
PGCP - FBD February 2026

Duration: 46 Classroom hours + 44 Lab hours

Objective:

1. To introduce the students to MongoDB, Node.js, Express.js, React, React-Redux, and practical relevance of all these technologies.

Prerequisites:

1. Any RDBMS (SQL), Web Programming Fundamentals (HTML, CSS, JS)

Evaluation method: Theory exam– 40% weightage

Lab exam – 40% weightage

Internal exam – 20% weightage

Text Book:

1. MERN Quick Start Guide – Build web applications with MongoDB, Express.js, React, and Node by Eddy Wilson Iriarte Koroliova / Packt

References:

1. Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node by Vasam Subramanian / Apress.
2. W3Schools Tutorials [<https://www.w3schools.com/>]
3. Curated Tutorial Links on ES6, React, etc. [<https://github.com/markarikson/react-redux-links>]

- **MongoDB: Data Modelling/DB operations**

Session 1 & 2:

- Installation of Visual Studio Code, Postman, Mongo DB & Node JS
- Relational vs NoSQL DB
- MongoDB fundamentals

Lab-1

Problem Statement 1:

Perform the following tasks on your system:

1. Install **Visual Studio Code, Node.js, MongoDB, and Postman.**
2. Verify the successful installation of each tool using appropriate **version or status commands.**
3. Create a simple project folder in VS Code and open it in the editor.
4. Run a basic **Node.js program** to print a message on the console.
5. Capture screenshots or outputs as proof of successful installation.

Problem Statement 2:

Perform the following tasks using **MongoDB:**

1. Explain the difference between **Relational databases and NoSQL databases** with one real-world example of each.
2. Start the **MongoDB service** and open the MongoDB shell or MongoDB Compass.

3. Create a **database** and a **collection** for storing student details.
4. Insert at least **5 documents** into the collection.
5. Perform basic operations such as **find, update, and delete** on the documents.

Session 3 & 4:

- Basic and advanced CRUD operations
- Aggregation pipeline
- Grid FS
- Replication
- Sharding

Lab-2 &3**Problem Statement 1:**

Perform the following tasks using **MongoDB**:

1. Create a database named companyDB and a collection named employees.
2. Insert **multiple documents** with fields such as employee_id, name, department, salary, and experience.
3. Perform **basic CRUD operations**:
 - Read employees belonging to a specific department
 - Update the salary of employees with more than 5 years of experience
 - Delete employees with experience less than 1 year
4. Perform **advanced queries** using conditions, sorting, and projections.
5. Verify and display the updated collection.

Problem Statement 2:

Perform the following tasks using **MongoDB**:

1. Use the **aggregation pipeline** to calculate:
 - Average salary by department
 - Total number of employees in each department
2. Store a large file (such as a PDF or image) in MongoDB using **GridFS** and retrieve it back.
3. Configure a **replica set** with at least three members and verify data replication.
4. Enable **sharding** on the database and shard the employees collection using an appropriate shard key.
5. Explain how **replication and sharding** improve availability and scalability.

- **Server side Development with Node.js and Express.js**

Module 1: Node.js

Session 5, 6 & 7:

- NodeJs architecture
- Modules: Built-in and custom
- Event loop
- Asynchronous application
- Testing node application

Lab-4, 5 &6

Problem Statement 1:

Create a **Node.js application** that demonstrates the core concepts of Node.js.

Perform the following tasks:

1. Explain the **Node.js architecture** with respect to single-threaded and event-driven behavior.
2. Create a Node.js program that uses **built-in modules** such as fs and http.
3. Create a **custom module** that performs a simple calculation and import it into the main program.
4. Demonstrate **asynchronous execution** by reading a file asynchronously.
5. Observe and explain the role of the **event loop** during execution.

Problem Statement 2:

Create and test an **asynchronous Node.js application**.

Perform the following tasks:

1. Build a simple **REST API** using Node.js.
2. Handle requests asynchronously and return appropriate responses.
3. Write basic **test cases** for the application using any Node.js testing framework (e.g., Jest or Mocha).
4. Execute the test cases and observe the results.
5. Explain how testing helps improve the reliability of asynchronous applications.

Module 2: Express.js

Session 8, 9 & 10:

- Introduction to ExpressJs
- Routing
- Template engines
- Middleware
- Web Application components
- Error handling

- Testing application Express application

Lab-7, 8 & 9

Problem Statement 1:

Create a simple **Express.js web application** that demonstrates core Express concepts.

Perform the following tasks:

1. Set up an **Express.js project** using Node.js.
2. Create **multiple routes** (/, /about, /contact) and return appropriate responses.
3. Integrate a **template engine** (such as EJS or Pug) to render dynamic HTML pages.
4. Implement **middleware** to log request details (method and URL).
5. Identify and explain the key **components of a web application** used in this project.

Problem Statement 2:

Enhance the Express.js application with **error handling and testing**.

Perform the following tasks:

1. Create **custom middleware** to handle **404 (Page Not Found)** errors.
2. Implement a **global error-handling middleware** for server errors.
3. Validate user input on one route and return appropriate error messages.
4. Write basic **test cases** to test Express routes using a testing tool (such as Jest or Mocha).
5. Execute the tests and verify correct responses and error handling.

● Front End management with React.js

Session 11 & 12:

- Introduction to SPA
- Component oriented architecture
- Introduction to JSX, Babel, Webpack
- React fundamentals
- Environment setup for react and react native
- React basics
- Styling react component

Lab-10

Problem Statement 1:

Create a **Single Page Application (SPA)** using **React**.

Perform the following tasks:

1. Explain the concept of **Single Page Applications** and **component-oriented architecture**.
2. Set up the **React development environment** using appropriate tools.

3. Create a basic React application with **at least three components** (e.g., Header, Content, Footer).
4. Use **JSX** to display dynamic content inside components.
5. Apply basic **styling** to React components using CSS.

Problem Statement 2:

Enhance the React application with modern tooling and structure.

Perform the following tasks:

1. Configure **Babel and Webpack** (or use a standard React setup that internally uses them).
2. Create reusable **functional components** using React fundamentals.
3. Pass **data between components** using props.
4. Apply different **styling techniques** (external CSS or inline styles).
5. Briefly explain how tooling like **Babel and Webpack** helps in building and optimizing React applications.

● **State Management with Redux**

Session 13 & 14:

- Introduction to state management
- Introduction to Redux
- Redux architecture
- Advanced Redux concepts and integration
- Basic and advanced Routing

Lab- 11, 12 & 13**Problem Statement 1:**

Create a React application to demonstrate **state management and Redux fundamentals**.

Perform the following tasks:

1. Create a React application with a simple **counter or todo feature** using **component state**.
2. Introduce **Redux** and explain why it is needed for state management.
3. Implement **Redux architecture** by creating **actions, reducers, and store**.
4. Connect the React application to the Redux store and manage the application state using Redux.
5. Display state changes in the UI and verify correct functionality.

Problem Statement 2:

Enhance the application using **advanced Redux concepts and routing**.

Perform the following tasks:

1. Implement **advanced Redux features** such as combining reducers and middleware.

2. Integrate Redux with asynchronous actions (e.g., fetching data from an API).
3. Implement **basic routing** using React Router to navigate between multiple pages.
4. Add **advanced routing features** such as dynamic routes or nested routes.
5. Ensure smooth navigation and state persistence across different routes.

Session 15, 16 & 17:

- React hooks
- React Context APIs
- Handling online payment
- Testing React components
- Firebase and React application
- Publishing the react application to AWS cloud

Lab-14, 15 & 16**Problem Statement 1:**

Enhance a React application using **React Hooks and Context API**.

Perform the following tasks:

1. Refactor an existing React component to use **React Hooks** (useState, useEffect).
2. Use the **Context API** to manage a global state such as **user authentication or theme selection**.
3. Share data across multiple components using Context without prop drilling.
4. Demonstrate state updates and side effects using hooks.
5. Verify the correct behavior through UI interactions.

Problem Statement 2:

Build, test, and deploy a React application with external services.

Perform the following tasks:

1. Integrate a **mock or sandbox online payment gateway** into the React application.
2. Connect the application with **Firebase** for authentication or database operations.
3. Write **test cases** to test React components using a testing framework.
4. Build the production-ready React application.
5. Publish the application to the **AWS cloud** and verify successful deployment.

- **Full stack application development using MongoDB, Express.js, Node.js and React.js**

Session 18, 19 & 20:

Module 1: Backend: Node, Express & MongoDB

- Create DB Objects

- Express JS Project
- Connect Backend Project to Mongo DB
- API Methods
- Upload File API
- Enable CORS
- Message Parsing Issue

Lab-17, 18 & 19

Problem Statement 1:

Create a **backend REST API application** using **Node.js, Express.js, and MongoDB**.

Perform the following tasks:

1. Create an **Express.js project** with proper folder structure.
2. Connect the backend application to **MongoDB**.
3. Create **database objects (schemas and models)** for a resource such as User or Product.
4. Implement **CRUD API methods** (GET, POST, PUT, DELETE) for the resource.
5. Test the APIs using **Postman** and verify database operations.

Problem Statement 2:

Enhance the backend application with advanced features.

Perform the following tasks:

1. Implement an **API to upload files** (such as images or documents) and store them appropriately.
2. Enable **CORS** to allow requests from a frontend application.
3. Handle **request body parsing** for JSON and form data and resolve any message parsing issues.
4. Add proper **error handling** for API failures.
5. Verify all APIs using Postman and document the responses.

Session 21, 22 & 23:

Module 2: Frontend: React JS

- Create React JS Project
- Routing and Navigation
- Bootstrap Integration
- Modal Popup Window
- Upload Image File
- Sorting and Filtering

Lab-20, 21 & 22**Problem Statement 1:**

Create a **React JS frontend application** for managing a simple product catalog.

Perform the following tasks:

1. Create a **React JS project** using a standard setup.
2. Implement **routing and navigation** to move between pages such as Home, Products, and Add Product.
3. Integrate **Bootstrap** to style the application layout and components.
4. Display product details in a tabular or card format.
5. Implement **sorting** of products based on price or name.

Problem Statement 2:

Enhance the React application with advanced UI features.

Perform the following tasks:

1. Implement a **modal popup window** to add or view product details.
2. Add functionality to **upload an image file** for each product and preview it before saving.
3. Implement **filtering** of products based on category or price range.
4. Ensure smooth navigation between components using React Router.
5. Verify the UI functionality and responsiveness.